ActInf ModelStream 010.1 ~ Building a Drone with RxInfer.jl ~ Bart van Erp

ModelStream | Building a Drone with RxInfer.jl ~ Bart van Erp, Albert Podusenko | 1:03:48

Hello and welcome. This is Active Inference Model Stream number 10.1 on April 4th, 2024, 4-4-24. And we're really excited today to be with Bart Van Earp and Albert Potosenko talking about building a drone with RxInfer. We have an ongoing RxInfer learning group at the Institute and a lot of participants have been excited to use even early versions of this package and connect it to all the exciting math and developments. So thank you both for joining, looking forward to this very pragmatic presentation. Awesome. Thanks, Daniel. Thanks for the kind words, the nice introduction. Also welcome everyone online. As Daniel mentioned, we're going to build a drone today. So last week I didn't know anything about drones whatsoever, but in the meantime, I was able to craft one with the help of my colleagues. And I want to kind of transfer the learning experience that I obtained over this week to all of you, such that you can leverage the power of our toolbox RxInfer in the end to make your own very cool applications, your own drone, your own autonomous vehicle, whatever you want. And we will then be there also to support you in this journey. But for now, let's get started. How are we going to build a drone with RxInfer? So first of all, let me briefly introduce myself. My name is Bart Van Earp. I'm a PhD student and teaching assistant in BIOS App, which is the research group here at Eindhoven University of Technology. And together with Albert, we will be also live and we'll also be monitoring the chat in case there are any questions pop up along the way. We will be your host today together with Daniel. This talk will be supported by Reactive Base. So Reactive Base is the overcoupling GitHub organization of RxInfer and all the packages on which it's built. At the moment, we're looking for contributors. So if you're interested after this talk, please reach out and then definitely we can get you acquainted with the toolbox and with our environment there. So we're from BiosLab, the Eindhoven University of Technology. We are located here in Eindhoven, also called City of Light. And from here, we're doing very interesting research on building agents, probabilistic graphical models and using these to craft all kinds of engineering applications, trying to solve real world problems with our engine.

Our engine, RxInfer, you might have already heard about this, but our engine is a Julia engine, which allows us to perform automatic Bayesian inference, specifically through reactive message passing. The engine that we have built is completely reactive, which means that it doesn't do anything unless there is something to do for it. So it's relatively energy efficient because it doesn't consume energy during all the other moments. And we perform message passing, but I'll give a more detailed discussion on this. More explanation will follow later in this talk. And we're going to use this toolbox in order to craft a drone, a drone that can actually fly using some simplified drone mechanics, go from position A to position B. And I'm very eager to show you how this journey, how this development journey looks like. So let's get started.

So what I'm going to talk about are three parts, basically, in this talk. I'm going to first start off with the model specification. So in the Bayesian inference methodology, the first part is always that we need

to craft a model. And this model will include some drone physics. So I'll be taking you back to high school, where you learned all these concepts, refresh a couple of them. And based on that, I'm going to construct a generative model of how a simplified drone would look like. Furthermore, I'll then continue upon the probabilistic inference. So if we have crafted this model, and we want this model actually to do something, inference allows us to compute posteriors in this model. And the inference on this model is actually the underlying control algorithm that we will be crafting. And this will be automated using our toolbox, RxInfer. Then finally, we'll wrap up with some experiments. And then there will be plenty of time to answer any questions that you might have. Let's get going. For the model specification, we're going to start off with some simplified drones physics. It will be very easy for some, a refresher for others. But hopefully, throughout this, throughout these following couple of minutes, you'll have a basic understanding of how a drone works. We have greatly simplified it. Because of course, going into the details of three, three dimensional drones, with wind, all that kind of stuff, would be a bit too much for just a single live stream. But we hope to offer you the tools and maybe the machinery to create these more complicated models yourself. So we have this very nice drone here. And what we're interested in is what forces act on this drone. So we're going to simplify this a bit. And we're saying that each of the rotors, so the left and the right rotor, produce some kind of upward force, pulling this drone from the from the ground. Of course, we also walk around on Earth, and we know that there is actually a force that opposes this, namely gravity, which tries to pull us to the ground. Well, combining these two forces allows us to either increase or decrease the altitude of this drone, and make it fly. That's the entire goal. In the end, the goal will be to come up with these forces, these FL and FR, such that our drone flies from location A to location B. If we do the computations a bit and see what are the kind of the net force forces acting on this drone, is in this case, we see that we have a vertical component, Fy, which is the sum of the individual forces created by the motor, minus the force of gravity that's acting upon the drone. And in the horizontal direction, because everything is nicely aligned,
we have a net force of zero. If there would be wind, for example, there would be a contribution there.
But in the simplified model, I assume that, well, there is no horizontal component as of yet.
But if we tilt the drone a bit with a certain angle theta, we actually see that this theta
not only affects, of course, the horizontal or the vertical net force, but also the horizontal one.
So based on this, we can obtain equations for this horizontal and vertical force acting upon the drone.
The motors, we assume that they are at a distance r, a radius r from the center of mass. So we very simplistically assumed that the center of mass of the drone is at the radius r from its rotor. So the force, you can think of it as acting upon an arm, creating a specific moment.
So the torque that gets generated by these rotors with respect to the center of mass of the drone,
the difference between the distance r is the difference between these two forces multiplied with the radius r.
With these three equations, so the net force is acting upon the drone, and this net torque that we can compute
based on the rotation that the drone wants to go to, we can already construct a big part of the model of this drone.

So these are kind of the forces that act upon the drone, horizontal, vertical direction, the rotation as given by this torque.

And we want to convert these forces into motion, because in the end we want to move the drone,

and we want to figure out what forces are required to move from a position A to a position B.

So let's see, using Newton's law, we can already know that our force is equal to our mass multiplied with our acceleration.

In other words, our acceleration is our force divided by our mass. So based on the net forces that we just computed, we can also extract equations for the accelerations in the vertical and horizontal direction.

These accelerations accumulate, so if we integrate over them, we get actually a velocity.

So an incremental change in our velocity you can think of is actually this acceleration times this incremental change in time,

which gives us kind of a relationship. So if we increase the acceleration over a period of time,

then we also know that that will lead to an increase in our velocities in both the horizontal and vertical direction.

If we take this even a step further, then we can also start to model the actual position of the drone.

So integrating over this velocity will give us our position in both the X and Y direction.

Basically every small change in X will be a result of the particular velocity over a small period of time.

And with this set of equations, it actually becomes possible to extract the movements from these net forces of the drone.

So that's basically the movement. So how it moves from left to right, up and down.

So we see this is the acceleration in the direction of

used for actually angular acceleration.

Similarly, we can also compute the angular velocity,

so omega, because a small increment in this angular velocity

will be our angular acceleration

times a small fraction of time.

So integrating over the angular acceleration

will give us the angular velocity.

And you might have already guessed this,

based on this, we can also compute a new angle.

So the angle is this velocity integrated over time.
Very similarly, as we derived the equations
for the actual movement of the drone.
And together, these net forces,
these equations for the movement of the drone,
and these simplified equations for the rotation,
they constitute the drone dynamics.
So it might have seemed like a recap from high school,
but for the simplified example,
this is all that we actually need
in order to craft this drone.
And that's what I'm gonna show you right now.
Because these equations of motion,
I'm gonna write down in a function,
specifically a Julia function,
because that's the language that RxInfer also uses
under the hood.
So we're gonna create this function F,
and I'm gonna walk you right through it.
There are just 15 lines of code,
so I think this should be manageable.
So we have a function F,
which accepts the state.
So you can think of this state as the positions,
velocities, angle, and angular velocities of the drone.
The actions, to which we refer to the forces
on the left and right rotor.
The particular drone,
like what are the characteristics of that drone,
and the environment.
So are we on Mars?
Are we on Earth?
All those attributes that are environment related.
So in the first three lines,
what we're gonna do is we're gonna extract some information.
So we're gonna extract the forces from our actions.
We're gonna extract the mass,
the moment of inertia,

and the radius of the drone,
because these are properties that correspond
to the particular drone that we're using,
and the gravity we're gonna just grab from our environment.
Because if we would be on Mars,
we would have very different dynamics, of course.
Then from the state,
what we extract are X and Y,
which are horizontal and vertical positions,
VX and VY,
which are velocities in the horizontal and vertical direction,
data, which was our angle,
and omega, which was our angular velocity.
With all these properties, characteristics,
and states extracted,
we can actually start implementing the equations
that we have on the left.
So FG, which is the force generated by the gravity,
is simply equal to the mass times the gravitational constant.
So FY and FX,
I merely copied from the equations on the left.
So it's the sum of the forces,
multiplied with cosine or sine of the angle,
and for the Y force,
the net Y force,
we also subtract the gravity,
because there it has an influence.
And for the net torque, we do the similar.
So we extract the difference between the force
generated by the left and right rotor,
and multiply that with the radius,
in order to get us a torque.
Then next up is we can, based on these forces,
we can compute the accelerations in the X and Y direction.
So F of X divided by M will give us these accelerations,
or F of Y divided by M for the vertical acceleration.
We're gonna take a very crude Euler approximation.
So we're gonna do a linearization

in order to actually create a new velocity.
So V X new, which will be the velocity after enforcing the actions,
will be our previous velocity,
plus our acceleration,
times this increment in time,
is DT.
And this DT we said beforehand.
But this is a simplified assumption,
which of course there are more complicated
assumptions to be made.
We could choose
Euler-Maiwanger,
and all those other fourth order approximations.
But for now, let's keep it simple,
and let's stick to this very simple order approximation.
Then based on the velocities and accelerations,
we can also do a similar trick for the new positions.
So how does our X and Y coordinate change based on their previous coordinates,
the velocities that we had at the previous moment,
and the accelerations that we computed?
Here we have this quadratic term over the accelerations,
which are merely a matter of fact of the approximation,
because over DT we also want to kind of integrate over the accelerations.
So that's odds, the second term basically,
odds for this effect and makes it a bit more realistic.
With that out of the way,
we can do similar things for the rotation.
So the acceleration, the rotational acceleration,
is our torque divided by our moment of inertia that we got from the drone.
We again can update the angular velocity based on this previous angular velocity.
The acceleration that we just computed and the time interval.
And based on that, we can also update our angle.
So what this function does,
it accepts the state actions and some characteristics of the drone and the environment
and some discretization over time.
And it computes and returns a new state.
So if we would have applied a specific action,
force FL and FR,

corresponding to the two rotor engines,
what would be the new state or approximately the new state that we end up with.
It's very simplified, but this you can think of as a state transition function.
With that out of the way, now it's actually time to start with the easy work.
Because we have now this function F,
and this function F specifies how we think this internal state of the drone evolves over time.
Something I forgot to mention, but Julia also supports Unicode.
So the people who are paying attention might have already seen it.
I still think it's a great feature,
but you can actually use Greek symbols,
which makes this translation going from physics to code way more easy,
and also allows you to retain this physical interpretation to it.
But that being said, let's continue with the proper model.
So just remember now that we have F, which encodes our state transition.
For our drone, we're going to construct a generative model.
And here I have drawn it as a so-called Forney-style vector graph.
But I'm going to talk you right through it.
So what we have is these boxes.
And these boxes correspond to factors or functions on how variables relate to each other.
Which one relates to another one?
And how are they interconnected through which functions and through which nodes, let's say.
And the edges, so the arrows here in this case, correspond to the individual variables in our model.
The variables S in this example correspond to the state of the agent.
So that contains the position of the agent in both x, y.
And the velocities also in the x, y direction.
Furthermore, this state also contains the angle and this angular velocity to make life a bit simpler.
U in this case corresponds to the actions.
So the actions that we want in the end to also infer.
Based on this figure, we can for example see around this function f,
which was our state transition.
That we put in the previous state.
And the actions.
And we get something out.
To that we add a bit of Gaussian noise to accommodate for any uncertainties that are maybe in the parameters.
In the environment because we use simplified drone physics.
Maybe we want to add a bit of process noise there.
Just to accommodate for the fact that these dynamics are not perfect.

And that there will always be internal, external fluctuations that drive this drone.

And that's the case.

So we can do that.

And we can do that.

If we write this down in a probabilistic manner, we can construct this particular vector graph.

Where we have these time slices which repeat over time.

And mathematically we can write that in the equation that's on the slide.

So we have a prior on the initial state.

Which we refer to as P of S0.

Which encodes our starting position, starting velocities, starting angles.

And all the way at the end of the model, we have our goal prior.

So P of ST.

Where capital T promotes the length of our model.

Also known as our horizon.

This P of ST describes, okay, where do we actually want the agent to go?

What do we want its velocity to be at the end?

What do we want its angle to be at the end?

All these kind of things.

So together, P of S0 and P of ST, these two priors,

de-encodes the starting and end position of our drone respectively.

Whereas this middle part with the F function, the transition function that we just described.

And the control U of which we can put a Gaussian prior because we want to infer it.

Together these encodes the dynamics of the drone, the generative model.

How does this look like?

I'm just going to port it directly into RxInfer.

So over the past couple of months, our colleagues have spent significant amounts of time

actually improving our modeling language.

Which makes it now even easier to craft your own probabilistic models in RxInfer.

And with these effectively four lines, five lines of code, we can construct the model, the graph that we have on the left.

So what I'm going to do is I'm going to talk you through it.

And I think at the end, I think, hope all questions or I don't think there will be a lot of questions surrounding this particular model any longer.

Because it's merely a replication of what's on the left.

We're going to define this variable S1.

And S1 in this case actually refers to the first state.

But Julia indexes with one.

So that's why it's not S0, but S1 instead.

And S1, which encodes our starting position, we're going to encode that with a multivariate normal distribution.

Whose mean is given by some initial state that we provide to the function.

And whose covariance matrix we assume to be very narrow.

Because we assume that we know the position of the drone at the initial time step.

Then inside this for loop, we're going to basically unroll this middle segment of the drone.

Of the generative model.

We're going to create a prior on the control.

So on this U of T.

And we're going to put there also a multivariate normal because that's nice to work with.

Whose mean is the mass times the gravitational constant divided by two.

And then as a vector of two.

The reason for this is because our prior assumption of this drone is that it exerts a force.

Because we think it's flying.

So it would make sense that we assume the force of the engines will be just enough maybe to balance the drone in the air.

So this mg divided by two, which is actually the force of the left and right rotor respectively.

They compensate for the force of gravity.

If the engine or the drone were to be upright.

So here we provide some additional information that allow us to improve convergence and improve also estimation.

Because we know that the drone is probably not going to drop out of air.

We want it actually to kind of stabilize and stay at its position.

So we do that here by putting this prior on the control.

Then following that is our state transition.

So how do we think our state evolves over time?

And this is where the dynamics of the simplified run that we talked about on the previous slide actually come into play.

So the mean of the next state will be given by this function f over the previous state and the corresponding actions.

And the drone, the environment and the discretization over time.

Together with some noise because of the unknown dynamics.

Maybe we've got some internal external fluctuations, uncertainties, stochastic behavior.

So that's all something that we also encode in this multivariate normal distribution.

So that incorporates this for loop.

And that this for loop basically unrolls this segment that's on the left containing this p of u, this f, and this norm.

Then finally we finish up our model by putting this goal prior at the end.

So at the horizon plus one because of the indexing, we're going to put the multivariate normal,

which mean is located at the goal containing our goal position, goal, the velocities that we wish to attain at the goal, the angle at the goal, etc., etc.

And together these couple lines of code, effectively there are just five, they create the generative model that's on the left.

So that's great.

We have crafted the model of how we think the drone evolves over time.

We have derived some simplified equations to model its states dynamics.

And now we want to do inference.

So we are interested in computing the marginal distributions over both actions and states.

Basically, we want to ask our model, okay, it's nice that you have encoded this starting and end position in the model.

But in order to obtain that, how can we actually reach that?

How can we actually do that?

So we're interested in figuring out what is actually the marginal distribution over the actions.

So this q of u.

And what is the marginal distribution over the states?

So what forces do we need to apply to go from position a to position b?

And how will this trajectory going from a to b look like?

In the general case, if we were to just try to solve this equation and compute these marginal distributions,

then we would have to integrate over all other parameters.

But that's very computationally heavy, because there are a lot of parameters in our model.

So doing this in a naive way would not be efficient at all.

But there is luckily a way to make it efficient, namely through message passing.

So let's dive a bit deeper into how that actually looks like.

So for this purpose, I've created this simplified factorized model to make sure everyone can kind of follow along.

So we have the different factors, fa, fb, fc, and fd.

Each linked to some of the variables from x1 ranging to x6.

And each factor is only connected to the variables to which it also relates.

It's a simple case, right?

Suppose now that we wish to compute the marginal here over the variable x4.

Doing this naively would require us to integrate out this factorized model over all variables x except for x4.

However, that's computationally heavy, as I mentioned earlier, because we have five variables to integrate over.

And that might be a challenge because we're in this very high dimensional space then.

Luckily, what we can do is we can make use of that these factors are not connected to everything.

So the factor nodes that are on the left are not connected to each of the variables.

And this turns out to be very beneficial from an efficiency standpoint.

So we can effectively split up this large integration over the entire model fx into a set of smaller integrals.

Simply by making use of this distributive property of integration where we can kind of separate them out, put some parts into brackets.

And by doing so, this large integration problem actually reduces into a couple smaller integration problems, which are more efficient to solve.

So we can also give an interpretation to this.

For example, this fa of x1, x2 integrated over the variable x1.

You can also think of this as closing the box, as we demonstrate on the left, over that factor fa.

And this is the result of this we call a message.

So this message you can think of as a summary of a particular part of that particular graph, of a part of the graph.

And we can iterate this a couple of times.

So this message mu of x2, if we propagate that into the next node, this fb, and we solve the corresponding integration with this double integration over now x2 and x3,

we get a new message mu of x4.

And we can do this on all the edges and over all the factor nodes here in our model.

We can integrate over the single node fd, over the node fc, where we have an incoming message mu of x5.

And what we end up with, and that's the interesting part, is that all these messages are relatively easy to compute.

They're relatively small because they do not require this huge integration.

The funny thing here is, is that this marginal, the one that we're interested in, this q of x4, is now simply the product of mu of x4 times the other mu.

So the messages that are actually colliding on that edge, and then of course renormalized.

But that marginal distribution can simply be computed on a very local level by multiplying the messages that are flowing over these edges.

Hence making it very efficient to solve this integration problem.

And this is also how we actually solve inference in our toolbox.

So in the probabilistic generative model that we created for the drone, we can use message passing to solve this marginalization in a very efficient manner.

To give you an example, these are some benchmarking results that we obtained with RxInfer.

So this is the final step of the model that we've done.

On the left you can see how long it takes for a simple stage-based model, which is similar to the model that I've shown before.

But you can think more of this like in a common filter setting for people who have a bit of familiarity with control systems.

And we can see that the filtering and the smoothing both scale very well.

They scale very nicely in the number of observation, or in the length of the model.

So just linearly basically.

On the right we also do a comparison between RxInfer, so R toolbox, and Turing.

Turing is another toolbox in Julia, which doesn't use message passing to solve the actual marginalization

problem.

But instead it reverts to sampling.

So instead of computing everything kind of analytically or mostly analytically or with approximations, it draws a lot of samples and tries to extract statistics from this.

But as you can see, the left axis is on the log scale.

So we obtain significant performance improvements in comparison to the sampling-based methods.

And that's also why we are pushing for RxInfer, because it's just way more efficient to perform these competitions with message passing in comparison to sampling-based methods.

There is one thing in our model.

So if you recall these drone dynamics, then our state transition was quite nonlinear.

So it was very challenging, or it is very challenging, to do exact message passing with this nonlinear node.

Because if we go from the previous state to the next state, then the distribution of this next state will no longer be nice in Gaussian.

But it might be a distribution which is outside of the distribution or outside of the family of exponential distributions, let's say.

So it requires some approximations in order to actually do inference around it.

So in the experiments, I have used unscented, which we can just specify with the little code block on the right.

And what this does, it draws a very small but very informative set of samples around the incoming Gaussian distributions,

which capture the first and second moment of that distribution.

So the mean and variances of these sigma points, these samples, these are still the same as the statistics of the incoming normal distribution.

What we then do is we use these very carefully selected samples to propagate through the nonlinearity.

And based on the transformed sigma points, we then reconstruct what the Gaussian at the output could have looked like.

So this unscented allows us to make approximations through this nonlinearity.

With that in place, we can actually perform inference.

So the code block on the left bottom is all you need in order to get the drone running.

So inside the in var function, we specify a model, which was the drone model that we specified in one of the previous slides.

We specify its data.

Although it's not observing something directly and it's more doing a planning task,

we can still supply it with the initial state and the goal state.

Where do we want the agent to move to?

And this meta object corresponds to the approximations that we wish to take.

And that's all.

That's all that's necessary in order to actually get this drone up and running.

And in order to demonstrate this, we have a very nice experiment created by Dimitri, one of our colleagues,

where we have this drone moving around.

But it will be, it's more complicated than, or I would say the plotting is more complicated than what I've shown on the previous slide.

Because actually getting this plotting to work requires way more code than actually getting our inference to work.

But what it allows us to do is not only to do this planning, but also to do this really in an online manner.

So here in this particular example, we have created this video where on the fly we are changing parameters.

So we're increasing the mass of the drone, making it heavier.

We're increasing the size of the drone, so it's radius.

It's engine power, like what are the limits of this drone, what can it obtain?

Gravity, perhaps you want to move to another planet and test your simulation there.

But we're changing these characteristics on the fly.

And with RX and Ferb, we are still able to process kind of this, do this planning task.

That being said, all the code and the examples that I showed you today,

they are publicly available on the reactive base organization.

So in the RX-infer-examples.gl repository, you can find the code required to run actually the drone experiment.

It will be a simplified experiment, so not the one on the previous slide,

because that would be way too intense to, that would be just too overwhelming to have a look at.

But the main characteristics of this particular drone demo are available here.

And if you have any questions, want to try it out, please let me know after,

or contact me if you have issues.

But the code that we developed for this particular live stream is actually publicly available.

And with that, I would like to thank you, of course, for this very nice opportunity, for this nice audience.

If you want to have more information, feel free to reach out to any one of us.

Reactive base, file issue, whatever you've seen deemed pretty.

You can contact us, research group, BiasLab, our spin-off, LACE Dynamics, for industrial applications.

But most importantly, for reactive base, I want to say that we are also looking for collaborators.

So if you got motivated by this talk, got inspired to create your own cool applications,

please reach out to us and hopefully help us develop RX-infer into the toolbox of the future.

I think that's something that I would like to strive for.

So if you're inspired, reach out and we'll make sure that there are amazing opportunities lying ahead.

And that's all. Thanks a lot for the attention.

And I hope it was an inspiring talk that got you engaged with RX-infer, got you excited.

So if you have any questions, please let me know.

Awesome. Thank you.

Very cool presentation.

Albert, do you want to give any first or overview remarks?

And then Chris and Kovac, we can discuss and ask some questions too.
Well, not really in a sense that, yeah, it was, although it was the first time I saw this presentation,
it was a great part.
I mean, I knew the underlying project behind that and how the drone works,
but the presentation, I think, was very nice.
It also connected how, well, the Bayesian inference is basically,
how is it connected to the goal-driven behavior.
I think it's really cool.
Yeah, I just wanted to add along the presentation that Bart was speaking about this noise component
when he drew, when he has drawn the graph.
So this noise component also could be seen as basically the compensation
for our poor approximation of the dynamics, right?
Because we use this first order linear approximation.
And, well, that's arguably not the best way of approximating the underlying physics,
although it worked for this simplified demo.
And there, well, I think I can just maybe add on how this demo can be extended
or this model can be extended.
Well, the first thing to do is, well, just to use a different approximation, right?
That's one thing that comes to my mind.
And another thing would be, and I think from the active inference perspective,
would be to omit some of the parameters in the transition function,
deem them unknown, and to let the agent figure out it
through the interaction with the environment.
So you could achieve that also with RxInfer
by putting some priors on the metrics
or putting some prior on the components of this metrics.
And you can see how fast the agent actually learns the physics of the environment.
Yeah.
Awesome.
I'll ask some quick questions from the live chat,
and then Chris and Kobus, if you want to ask, feel free.
Okay.
So Carlos asks,
is the reactive part based on events to trigger the pipeline or based on signals?
Yeah.
Yeah, go ahead, Bart.
Yeah.
So what we, or what Dimitri did when he built it,

when he started building this RxInfer toolbox,
is he used a very different computing paradigm.
So normally you execute code from the top to the bottom.
But what he did is he used this reactive paradigm.
So based on events,
a particular computation is being executed.
And these events can be various.
It can change, of course.
It depends very much on the context what an event is.
But often the event corresponds to making actual observations.
So especially for active inference agents,
if we make a new observation,
only then re-computations are being executed.
So basically the entire algorithm does nothing
unless there is something to compute.
And I hope that kind of answers the question,
but I'm not entirely sure.
So if I was a bit unclear,
maybe Albert, you can add to it if you have a better.
Yeah, I'm just not sure which kind of reactive part was meant in the question.
Because yeah, there is the reactive part of the inference,
but also the demo itself was reactive, right?
Like when you are triggering this.
But basically speaking about how it works,
indeed it triggers on the event, basically.
So in the event is that, yeah,
there are a few things that could be changed, right?
It's either environmental change,
something changed in the environment,
and then we sense this from our sensors,
and then the inference follows, right?
So there is, just to add upon what Bart said about this reactive part,
so message passing is a well-known algorithm.
I mean, it's been around since Perl,
or I believe propagation, perhaps later, earlier.
But then the reactive is a particular implementation of this message passing,
which is more kind of driven by how the system should work in the field.
Awesome.

Yeah, with Magnus Kudal, we explored a lot of the message passing,
talked a little bit about how all the messages could be passed at once,
or the reactive paradigm where you could uncouple the rates of updating.
Okay, I'll ask one more question from the live chat,
and then Chris or Kobus.
Fraser asks,
Fantastic stuff!
Double exclamation point!
Uh-oh.
If somebody wanted to help develop RxInfer with your organization,
what are the ideal characteristics and backgrounds to have?
I think we are very open.
So also within our lab,
we have a huge variety of backgrounds,
ranging from computer science to physics,
electrical engineering for myself, for example.
So there is a wide variety of backgrounds possible.
Of course, we assume that,
or we would like coding to be something that you're good at or willing to learn,
because I think that's one of the most important things,
of course, to develop a software engine.
And all the rest,
I think we can provide you with excellent materials to get crossed,
or to get familiar with the methods that we use,
or get acquainted with the models that we have developed in the past.
So,
personally,
when I started in this group,
and when I started working on this project,
I also didn't have any background.
So I used to be an air electrical engineer.
I never heard about,
well,
I was taught probability theory,
and then quickly forgot about it.
So my background was very different.
And it's maybe a bit difficult to get started,
started directly with all this probability message passing,

and all this entire paradigm.
But it's definitely a very nice ride to learn.
So,
I would say,
if you're willing to learn,
willing to make a nice contribution,
then that's always welcome.
Yeah.
Yeah.
Well,
we are mostly writing stuff in Julia,
right?
So Julia is sort of,
I mean,
I mean,
it's learnable language,
right?
It's not like,
if you don't know Julia,
you can't help us.
No,
that's not,
absolutely not true.
So we welcome people from different programming backgrounds.
And besides,
there are different branches on how you can help within this Rx-Inferrin,
basically,
the whole reactive base ecosystem,
because it consists not only of the inference engine,
which I must say,
it's the most difficult part for contributing,
because there are many intricacies involved within this inference language.
But there is also GraphPPL,
right?
There's also Rocket.
There are other packages which basically constitute the whole ecosystem.
And yeah,
well,

you don't have to be an expert in Bayesian inference to contribute to Rocket,
or you don't have to be,
again,
an expert in particular approximations to contribute to a model builder.
So,
and of course,
the documentation is something which always needs an update,
right?
So we don't dismiss people for contributing to the documentation.
That's extremely important for the community and for ourselves,
because we can see,
okay,
so where are the parts which we could explain better or actually improve our
ecosystem?
Awesome.
Kopus?
Very interesting talk.
Thank you very much.
I have a question about categorical examples,
where the control space consists of values from a categorical distribution.
So,
does RxInfer have any such examples available?
We do.
We do.
So,
we do not only support continuous variables in this case,
but we also support a variety of categorical,
yeah,
Renouli distributions,

The categorical distributions themselves.
We do have a lot of examples.
So,
if you visit the website,
rxinfer.ml,
there is this huge list of examples,
use cases,
some of which feature categorical distributions as well,

modeling as,
yeah,
all the rate,
ranging from modeling as simple coin tools,
to more advanced examples.
But we do offer functionality in both the discrete domain,
as in the continuous domain.
Yeah.
Okay.
Thank you.
Yes.
Yeah,
go ahead.
Go ahead.
Now,
I just wanted to add,
there is,
well,
the most,
kind of,
famous example on,
or,
hello world examples,
into the categorical world within,
I think,
is,
is hidden market model.
And I think we have an example on that.
I'm not sure that we have incorporated the control within that example.
That's probably learning the transition matrices for observations and,
and,
well,
the hidden state.
But there is,
I think,
nothing stopping you from introducing the control for,
for a certain,
discrete type of Markov model.

Thank you.
And then I use VS code.
And I do have problems with Greek symbols.
When they contain multiple characters in a superscript or subscript,
for example.
So do you guys use VS code at all?
And if so,
do you have a best practice to,
to solve my problem?
We do use VS code.
I think our entire lab uses VS code because it does turn out to be a very,
very nice tool.
So it's good that you mentioned this.
Regarding the subscripts and the superscripts that you mentioned is,
personally,
I try to avoid them because oftentimes I completely agree that it looks nice to
have this dot over variable,
for example,
to denote the derivative or to draw kind of the index with a superscript.
But personally,
to me,
that clutters it a bit and also makes it more challenging to edit.
So I just try to refrain from using sub and superscripts in VS code with this
auto tab,
although it looks very cool and might be nice for like demo purposes or actually
experimentation or development.
I always try to avoid them.
So not a solution,
but this is my kind of my take on it.
Okay.
Yeah,
I can understand.
Yeah,
because,
and another possible problem with super and subscripts is they can appear very
small and I often find myself having to zoom in a bit to,
you know,
discriminate between symbols.

And then my final question,
to what extent can you guys parallelize within RX infer?
Are there obvious low hanging fruit choices to,
to apply parallelization?
In RX infer?
Yes.
So,
um,
it's good that you mentioned this because at the moment we have actually a
master's student who is kind of working on this particular,
or trying to work on this project,
uh,
where we try to parallelize the message passing and the computation,
for example,
of the marginals.
So there are opportunities there to do some sort of parallelization.
Um,
in general,
at the current moment,
our message passing is all linear,
unfortunately.
However,
there have been papers where they also parallelize that,
although it seems that it's impossible to do.
Apparently it's still this,
uh,
but that's something perhaps for a future project.
So at the moment we do not provide a lot of parallelization around RX infer,
but in the future,
this is definitely on top of our agenda to make also improvements,
uh,
in terms of performance and in computational speed.
Um,
uh,
just one quick question,
um,

another one,

um,
have you guys thought of approaching,

big players like Amazon web services,
for example,
to,

you know,
and present this to them and try and get them to incorporate it into their,
into their tools.
Um,
we have been,
um,
so with Lace dynamics,
kind of the spin off company,
which also uses RX infer to develop real applications,
let's say for customers,
we have considered this,
this option.
Um,
but at the moment we have not been in contact to any of these,
these companies or clients.
Uh,
sorry.
No,
I just wanted that to,
we,
we haven't talked to none of,
uh,
Fang.

so that's,
that's,

yeah,
we haven't done yet.

we are,

we are in talks.
There are a few interested parties in the pipeline,
but,

uh,
no,
we haven't approached Amazon.
No,
not yet.
Excellent.
Thank you.
Kobus.
I'll ask another question in the chat and then Chris,
if you have anything,
Fraser asks,
what are the largest current challenges to the reactive message passing paradigm
for doing this kind of approximate Bayesian inference?
It's a very good question.
I think it also depends a lot on who you ask this question to,
to me personally,
um,
the message passing,
I think is a great method in itself.
And it's also overcomes already a lot of challenges.
What I think would be the most fruitful next step is to really start working on the actual modeling,
the modeling aspects.
So can we create maybe universal models,
uh,
hierarchical models?
How can we do this such that these hierarchical models or similar actually solve more complicated tasks than a simple stage-based model can do.
Um,
so for me personally,
I think that's where we can achieve the biggest gains in this modeling part.
I'm not sure about what Albert thinks are the most big,

what is the,
yeah,
what the biggest outstanding challenge is,
but there are,
I think,
I think from,
from,
yeah,
totally agree.
from the,
from the research perspective,
the structural adaptation of,
of the model,
that's the most,
I would say,

the hardest and the most interesting,
uh,

uh,
from my opinion,

problem.
That's like hand crafting this model,
like for,
for this type of drones,

It's nice.
It can be easy.
Sometimes not.
Well,
if you go to the three dimensional,
that's significantly,
uh,
more,

complex.
indeed like having,

um,
a machinery.
Well,
the thing,
the interesting thing that we,
we do have a machinery to,
to do a structural adaptation,
uh,
because,
uh,
Rix and Ferb,

does provide the,
uh,
an opportunity to extend your model,
to patch it.
Uh,
although how to do that in principle,
how do you indeed,
uh,
grow or,
or,
uh,
shrink your model.
And so in time while observing and being,
being in the field,
uh,
I think this is the most challenging part indeed.
And as for,
well,
there are certainly,
uh,
problems with,
uh,
approximations and,
uh,
uh,
well,

universal rules.
Uh,
that's something we also,
I think,
uh,
outline in a few demos that,
well,
there could be a situation where the rule for computing the message,
which,
uh,
Bart was showing is not,
is not available.
Uh,
so there is always this,
uh,

right?
to,
uh,
well,
to commit to a simpler approximation to do it faster.
You want to commit to,
uh,
uh,
more accurate approximation,
but it will take more,
uh,
resources,
uh,
computational resources and how to balance between these two.
That's,
that's another interesting,
uh,
question.
How to,
how to automatically decide.
Uh,

I think,

I think this is,

uh,

what I would call,

um,

I would also add from our kind of educational and research side that one of the big challenges,

thankfully one being approached is to develop the documentation and the examples.

The examples help us learn, but also a lot of us are having good success bringing in code examples into code and language models and using the working examples to template new examples.

So as the library of open source published models grows, that will become more possible.

And then also there's a lifelong, fascinating journey of talking about how do you go from seeing a drone in the sky to getting those physics equations down and that kind of like approach to modeling.

And how do we get to a graph, assuming that that's where the software package can pick up and render it kind of no problem.

But there's still this very human, very collaborative process, which also could receive a lot of documentation and examples about getting to that kind of a model.

Chris?

Yeah, absolutely.

Yeah, yeah. Chris, do you want to ask anything?

No, I just wanted to say thank you guys very much. I really enjoyed the presentation.

I liked, I really enjoyed how you broke down a lot of the top of my head right now is like the integration example.

It was very intuitive. It made a lot of sense and it really highlighted a lot of the benefits and true power in what you guys have developed.

So thank you guys. And I really, really appreciate your time and explain this to us.

Thank you for the kind words.

If I could also add one comment, just as we've been working on it week by week, a real like arising insight that got us both excited and not daunted,

but just realizing the scope of the package and the work was in comparing the Rx and FUR active inference examples

with other active inference simulations like from PyMDP or built kind of custom outside of a package like the In4ANCE examples.

And we realized that a lot of alternative approaches, people try to go as fast as possible to developing an active inference agent

and then talk about like the variational free energy or the expected free energy of the agent.

And so the package serves as a helpful accelerator to get to an agent, but that's not like the general agent.

And then even making small changes to that scheme can be buried and scattered across different packages.

And then there's all these other kinds of limitations that one quickly finds themselves encountering like in the education application settings.

Whereas in Rx and FUR, it's such an engineering driven approach.

So a lot of the engineers on our team were immediately more comfortable.

And then a lot of people who had been playing with active inference from a computer science or from a philosophical and mathematical side

were a little bit surprised because we were getting very low level with the node engineering.

And then after wiring that up and understanding it, it's just like press play.

So a lot of the focus on developing an agent was actually that focus was moved to better understanding the graph

that represents the ecosystem of shared intelligence.

And there was less upfront focus on merely making it like an agent-based model

because you could make an agent-based model like the drone,

or you could develop other kinds of graphs that don't necessarily have an agentic basis.

So that helped us realize that this is a much broader toolkit than doing active inference or agent-based modeling of any kind,

but that those are use cases that are benefited greatly by it.

Definitely.

I think you are...

Actually, the main point of our package.

So RX-Refur allows us to enable or enables us to create these agents very quickly, very adapt, very efficiently.

So if we compare this, for example, to...

And you mentioned this already also from your engineers.

If you would go through a traditional design cycle for an engineering firm, you would create a single model kind of thingy,

a single algorithm, let's say 50 pages of code or something.

And that's nice, these 50 pages of code, but it's unmaintainable.

You need a lot of engineers to actually make improvements upon it.

Yeah, if something changes, you don't know what's going to happen, right?

Whereas with RX-Refur, we just have to specify this little piece of code which specifies the model.

And then inference is automated, as you mentioned, you just press play.

It isn't that easy, of course, from the actual engine perspective.

But we try to make the user experience as good as possible to make it as easy on the user to build agents, active inference agents,

or any other applications where you might need such machinery to actually complete a specific task.

And hopefully by doing so, by making this user friendliness, one of our priorities is we can also enable active inference

getting adopted across industries and across research groups.

Yeah.

After today, if we have a quick question, is there an email that we could send those questions to?

I think from the Active Inference Institute, there is a communication channel.

But Daniel, please correct me if I'm wrong, directly to BiasLab.

But if you have questions, feel free to start a discussion on GitHub because we actively maintain it.

We keep also a close watch on what's happening.

And there might also have been people with similar questions before you.

So my first hunch would be to create a discussion on GitHub.

And we will definitely then get involved with one of our colleagues in order to help you out.

Okay.

Thanks.

Well, if you have questions regarding Easy Dynamics venture, then you could use our info at lacedynamics.com as well.

And also just to dump any other questions you want.

Yep.

Cool.

Well, thank you all again.

It's really awesome.

I was thinking about how the packages are kind of factorized and modular and separable, almost like that's a theme that helps ecosystems work.

So overall, thank you very much for the work and for presenting.

We've really appreciated the chance to work with a package in our learning groups.

And we'll look forward to seeing more examples and sharing more also from the projects that people are working on in the Institute.

Thank you, Daniel.

Thank you very much.

It was a pleasure.

Thank you for having us.

Yeah.

Okay.

Great.

So see you all next time.

Bye.

Thank you.

Bye.

Bye.